07 · Interpretability

Siddharth M. Bhatia · siddharth@sidmb.com
July 1, 2026

Contents

Activation capture 1
Build a probe direction from captured activations 2
Steering — add or ablate a direction 2
Activation patching — cross-branch causal tracing 3
Token logprobs / surprisal / entropy 4

All four tools hook into the **same** generation path as real turns, `sample`, and tool loops, and are tagged to conversation structure. They apply to **local `ModelParticipant`s only** — an `APIParticipant` raises on any interp request rather than silently no-op'ing. Pass them to `conv.step(...)` or `conv.sample(...)`.

Activation capture

`conv.capture(...)` is a context manager: every `step`/`sample` inside the block records activations into a fresh `ActivationCache`, auto-tagged by speaker + turn.

```
with conv.capture(sites=["residual"], layers=[8, 12, 16], offload="cpu")
     as cache:
    conv.step(conv.participant("bob"))
    conv.step(conv.participant("alice"))

# Query the cache by structure:
act = cache.at(participant="bob", layer=12, site="residual")   # exactly
     one tensor, [seq, d_model]
records = cache.query(participant="bob")                        # all
     matching ActivationRecords
```

- **`sites`**: `"residual"` (per-layer residual stream), `"attn"` (attention sublayer output, post-o_proj), `"mlp"` (MLP output). Capture is a single clean forward over the full prompt+generation.

- **`layers`**: a list, or `None` for all layers. Keep it **narrow** — all layers × all tokens × many rollouts OOMs fast.
- **`offload`**: `"cpu"` moves tensors off-GPU as captured (essential for sweeps; uses a batched pinned transfer under the hood — ~7× faster than naive `.to('cpu')`); `None` keeps them on-device.

Each `ActivationRecord` carries `.participant`, `.message_idx`, `.layer`, `.site`, `.tensor` (`[seq, d_model]`), `.token_span` (prompt_len, seq_len), and `.phases` — a `{prompt/reasoning/answer: (start, end)}` map so you can slice reasoning-vs-answer activations for CoT models:

```
rec = cache.query(participant="bob", layer=12)[0]
a0, a1 = rec.phases["answer"]
answer_acts = rec.tensor[a0:a1]        # activations over just the answer
     tokens
```

Build a probe direction from captured activations

```
import torch
# e.g. mean residual at layer 12 over the answer span for two conditions →
     a difference-of-means direction
pos = cache.at(participant="bob", layer=12)[rec.phases["answer"]
     [0]:].mean(0)
# (collect a `neg` from another branch/condition similarly)
direction = (pos - neg)
direction = direction / direction.norm()
```

Steering — add or ablate a direction

`SteeringSpec` registers forward hooks that either **add** `coef * direction` to the residual at `layers`, or **ablate** (project out) that direction.

```
from interlens import SteeringSpec

steer = SteeringSpec(direction=direction, layers=(8, 12), coef=6.0,
     mode="add")
msg = conv.sample("bob", "How do you feel about the proposal?",
     steering=steer)  # ephemeral, steered

ablate = SteeringSpec(direction=direction, layers=(12,), mode="ablate")
     # remove the component
```

```
conv.step(conv.participant("bob"), steering=ablate)
     # committed, ablated turn
```

`direction` is a `[d_model]` tensor on any device (moved to match). A summary (mode, layers, coef, direction norm) is recorded in `msg.metadata["steering"]` so a steered turn is reproducible. Steering **disables KV reuse** automatically (the intervention wasn't in the cached KV).

Activation patching — cross-branch causal tracing

`Patch` overwrites a decoder layer's residual at specific token `positions` with activations captured elsewhere (e.g. from another branch). This is the causal-tracing primitive: capture at turn N in one branch, inject at the aligned positions of another branch's forward.

```
from interlens import Patch

# 1) capture bob's layer-12 residual in the clean branch
clean = conv.branch()
with clean.capture(sites=["residual"], layers=[12]) as cache:
    clean.step(clean.participant("bob"))
donor = cache.at(participant="bob", layer=12)          # [seq, d_model]

# 2) inject those activations at chosen positions in a corrupted branch's
     next forward
positions = (3, 4, 5)
patch = Patch(activations=donor[list(positions)], layer=12,
     positions=positions)
corrupt = conv.branch()
corrupt.transcript[-1] = ...                            # (corrupt the
     setup somehow)
patched_msg = corrupt.sample("bob", patch=patch)       # bob generates
     with layer-12 residual patched
```

Aligning positions across branches is the caller's responsibility; `Patch` just performs the overwrite (only on the prefill forward, not per-token decode). Patching also disables KV reuse.

Token logprobs / surprisal / entropy

```
msg = conv.sample("alice", "State your final answer in one word.",
     return_logprobs=True)
md = msg.metadata
md["logprobs"]    # per generated token: log p(token)
md["surprisal"]   # -logprob (nats)
md["entropy"]     # full next-token distribution entropy at each step
     (model uncertainty)
```

These are scalar-per-token lists (safe to store in metadata). Works with `step` too.

Next: advanced pipelines that combine these, or scaling to rollouts.

© 2026 Siddharth M. Bhatia. All rights reserved. Interlens is licensed under the GNU Affero General Public License v3.0.